CS61C: Great Ideas in Computer Architecture (aka Machine Structures)

Lecture 12: SDS Part 2

Instructors: Justin Yokota

Agenda

- Simplifying Circuits
  - Boolean Logic
- Finite State Machines

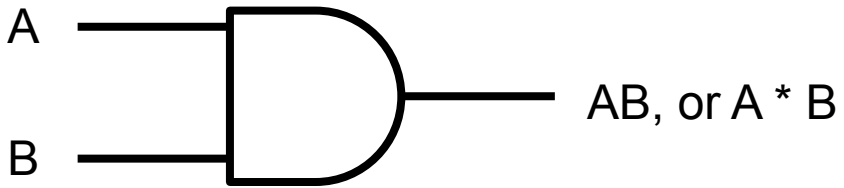
Agenda

- **Simplifying Circuits**
  - Boolean Logic
- Finite State Machines

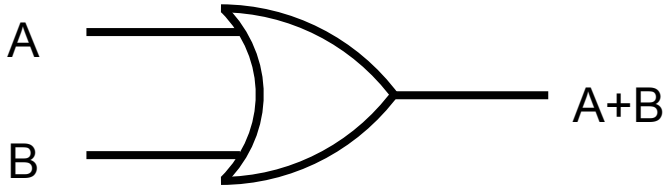
Boolean Algebra

- Every function from $\{0,1\}^N \rightarrow \{0,1\}$ has a corresponding truth table
- Truth tables often have multiple circuits and logical expressions that compute their function
- Need standardized way to notate boolean expressions.
- Boolean Algebra: Use math notation to write basic logic operators (OR, AND, and NOT)

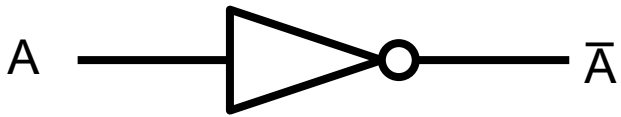
Boolean Algebra



AND: Notated as multiplication



OR: Notated as addition



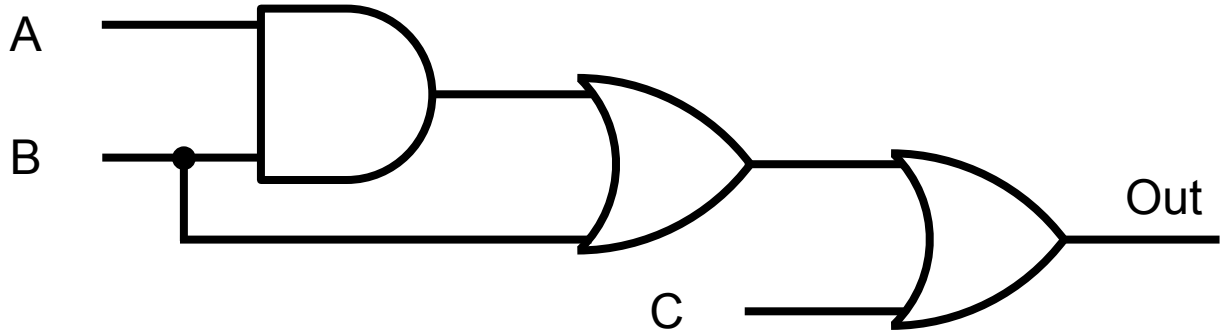
NOT: Notated with overline

1 = TRUE, 0 = FALSE

Boolean Simplification Rules

AND Form	OR Form	
$x \cdot y = y \cdot x$	$x + y = y + x$	Commutativity
$(xy)z = x(yz)$	$(x + y) + z = x + (y + z)$	Associativity
$x \cdot 1 = x$	$x + 0 = x$	Identity
$x \cdot 0 = 0$	$x + 1 = 1$	Laws of 0's and 1's
$xy + x = x$	$(x + y)x = x$	Uniting Theorem
$x(y + z) = xy + xz$	$x + yz = (x + y)(x + z)$	Distributivity
$x \cdot x = x$	$x + x = x$	Idempotence
$x \cdot \overline{x} = 0$	$x + \overline{x} = 1$	Inverse (Complement)
$\overline{(xy)} = \overline{x} + \overline{y}$	$\overline{(x + y)} = \overline{x} \cdot \overline{y}$	DeMorgan's Laws

Can we make an equivalent circuit with fewer gates?



Equivalent Boolean Expression:
((AB)+B)+C
= B+C

Simplifying Sum of Products

$\bar{A}\bar{B}C+\bar{A}B\bar{C}+\bar{A}BC+A\bar{B}C+AB\bar{C}$ (21 gates)
$\bar{A}\bar{B}C+\bar{A}B(\bar{C}+C)+A\bar{B}C+AB\bar{C}$
$\bar{A}\bar{B}C+\bar{A}B(1)+A\bar{B}C+AB\bar{C}$
$\bar{A}\bar{B}C+\bar{A}B+A\bar{B}C+AB\bar{C}$
$\bar{A}(\bar{B}C+B)+A\bar{B}C+AB\bar{C}$
$\bar{A}(\bar{B}C+B+BC)+A\bar{B}C+AB\bar{C}$
$\bar{A}(B+\bar{B}C+BC)+A\bar{B}C+AB\bar{C}$
$\bar{A}(B+(\bar{B}+B)C)+A\bar{B}C+AB\bar{C}$
$\bar{A}(B+C)+A\bar{B}C+AB\bar{C}$
$\bar{A}(B+C)+A(\bar{B}C+B\bar{C})$ (10 gates)
Can we do better? Yes (with different simplification order)
Best I could find: (B+C)(!(ABC)) = 5 gates

A	B	C	Out
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	0

Bad News

- Is it possible to algorithmically determine the fewest gates for a given Boolean Expression?
  - No known algorithm in polynomial time
    - If you do find one, P = NP
- Does a randomly generated circuit have a small circuit?
  - No; Sum of Products is asymptotically optimal in expectation

Good News

We're not dealing with random Boolean expressions!

- Mathematical operations have meaning behind them
  - Tends to lead to simple circuits
- The operations we mandate in RV32 base are precisely the ones with simple circuits
- RV32's instruction translations are chosen specifically to simplify the circuit

Often it's possible to find "patterns" in a truth table and get a small circuit that way

In most cases, we don't expect you to get the absolute smallest circuit for a given truth table

Simplify the truth table (Out0 and Out1 equations)

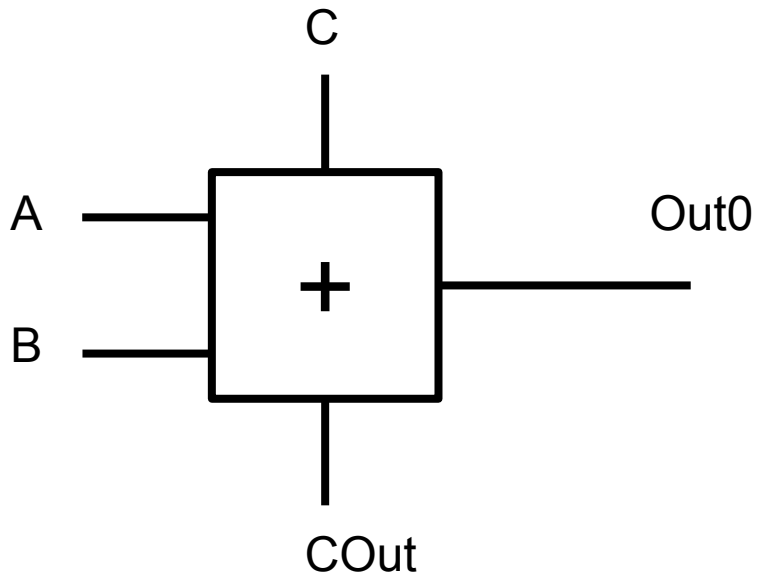
You may use XOR, represented by ^

But that uses however many gates you used earlier to make XOR (OR, AND, NOT count 1 each)
SOP score: 31 total gates
Gold Medal: 15 total gates
Platinum Medal: 12 gates

A	B	C	Out1	Out0
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

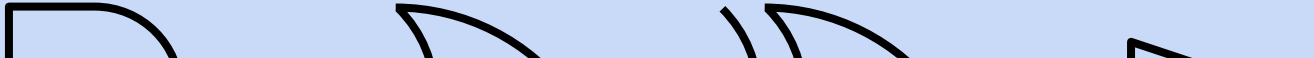
Simplify the truth table (Out0 and Out1 equations)

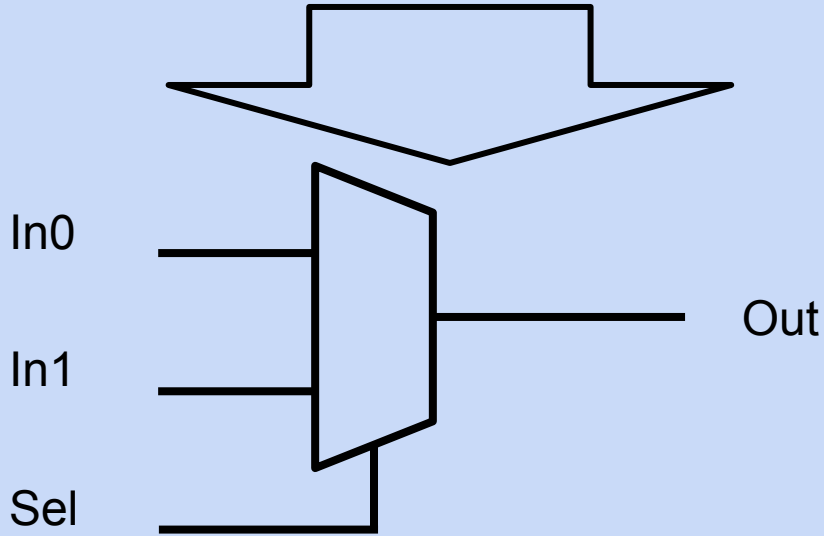
This circuit is known as a "full adder"
(Adds A+B+C)



A	B	CIn	COut	Out0
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

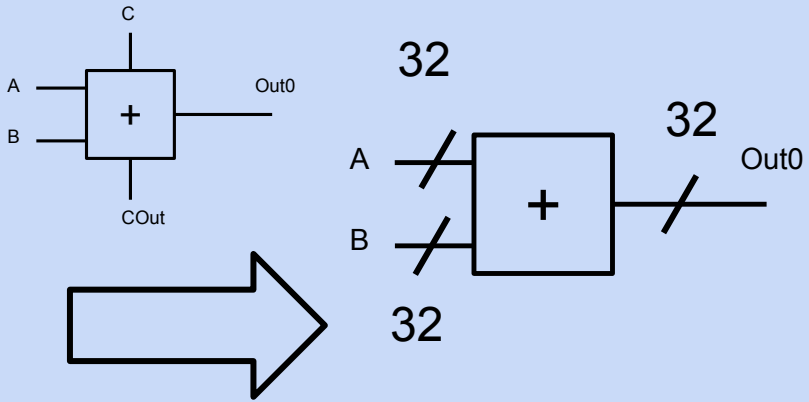
Using AND, NOT, OR, and XOR, create a MUX gate





Sel	Out
0	In0
1	In1

Create a 32-bit Adder using this subcircuit



SOP Score: ~32 sextillion
Gold Medal: 500 gates
Platinum Medal: 360 gates

A	B	Out0
0x00000000	0x00000000	0x00000000
0x00000000	0x00000001	0x00000001
0x00000000	0x00000002	0x00000002
0x00000000	0x00000003	0x00000003
…	…	…
0xDEADBEEF	0x12345678	0xF0E21567
…	…	…
0xFFFFFFFF	0xFFFFFFFF	0xFFFFFFFE

Scratch Space

Agenda

- Simplifying Circuits
  - Boolean Logic
- **Finite State Machines**

Finite State Machines

- A simplified computing model
- Variations exist in other classes
  - CS 70 talks about probabilistic FSMs (Markov Chains)
  - CS 172 goes into DFAs and NDFAs
- If you know about FSMs from other classes, the variant we use in this class:
  - Receives one input per cycle
  - Outputs one value per cycle, instead of having a set of accepting states
  - Starting state, no stopping states

Finite State Machines

- A Finite State Machine consists of a set of **states**

Finite State Machines

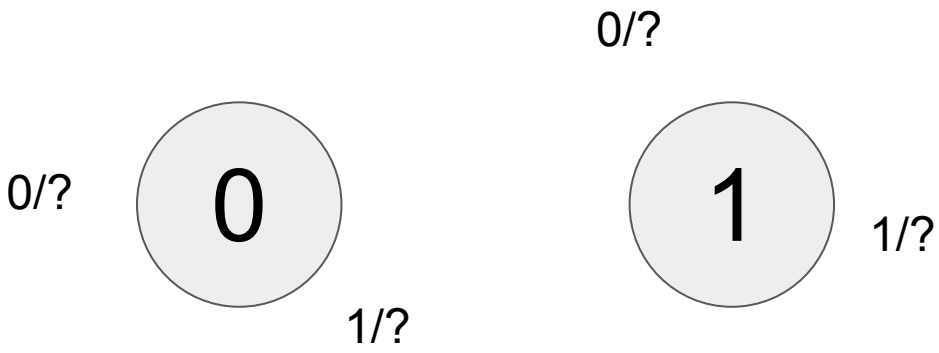
- A Finite State Machine consists of a set of **states**
- Computation is done in **clock cycles**

Finite State Machines

- A Finite State Machine consists of a set of **states**
- Computation is done in **clock cycles**
- Every clock cycle, the FSM receives an input

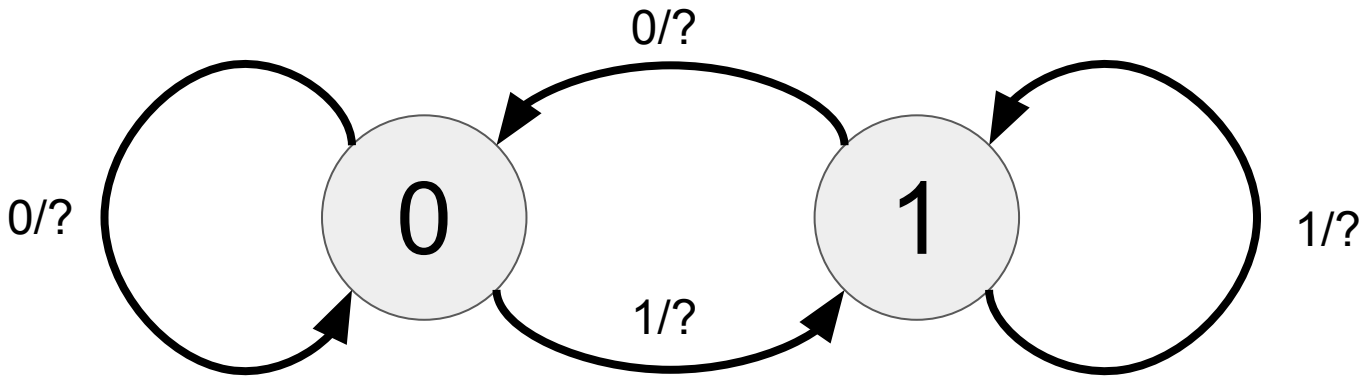
Finite State Machines

- A Finite State Machine consists of a set of **states**
- Computation is done in **clock cycles**
- Every clock cycle, the FSM receives an input
  - Based on the input and its current state, it:



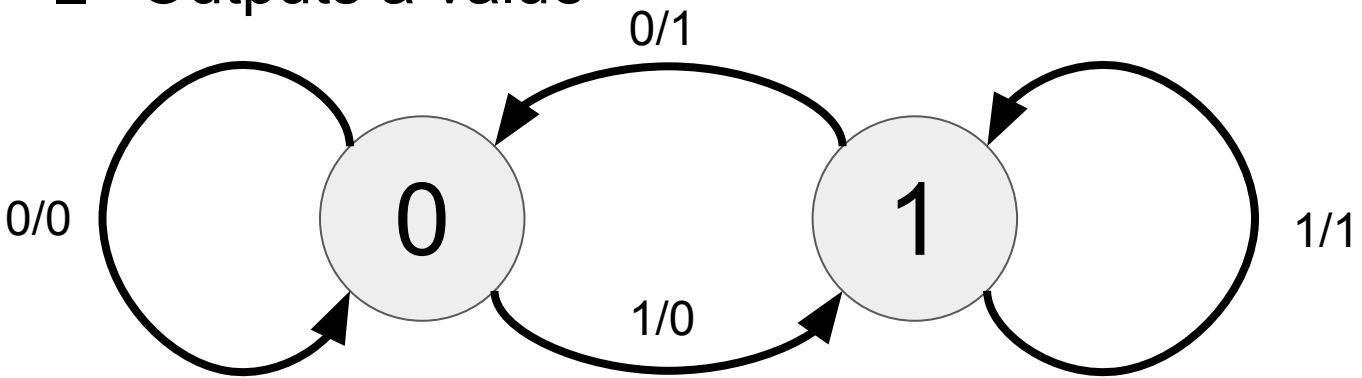
Finite State Machines

- A Finite State Machine consists of a set of **states**
- Computation is done in **clock cycles**
- Every clock cycle, the FSM receives an input
  - Based on the input and its current state, it:
    - Moves to a new state (potentially its own)



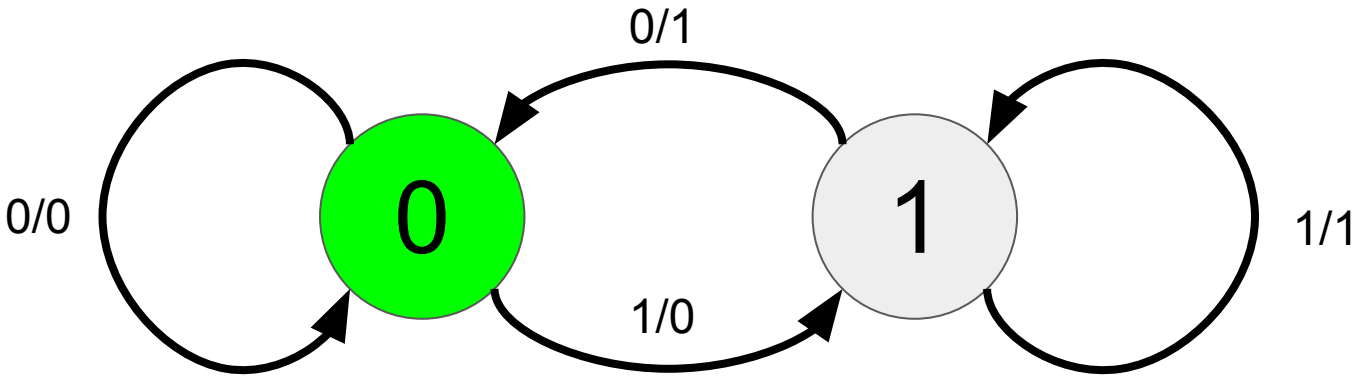
Finite State Machines

- A Finite State Machine consists of a set of **states**
- Computation is done in **clock cycles**
- Every clock cycle, the FSM receives an input
  - Based on the input and its current state, it:
    - Moves to a new state (potentially its own)
    - Outputs a value



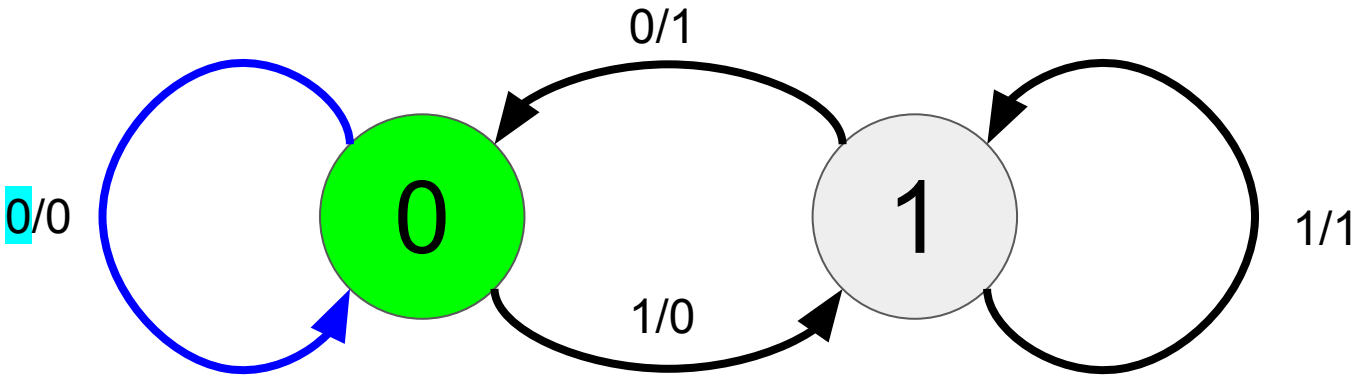
Finite State Machines

Example: Say we start at state 0, and send this input:
0b01101101101110111000101 10
What is the resulting output?
0b



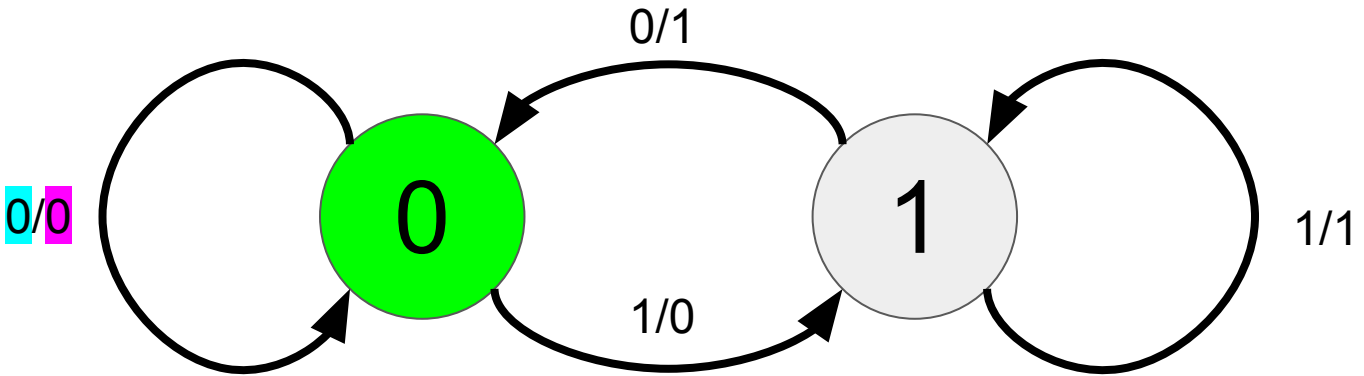
Finite State Machines

Example: Say we start at state 0, and send this input:
0b01101101101110111000101 10
What is the resulting output?
0b



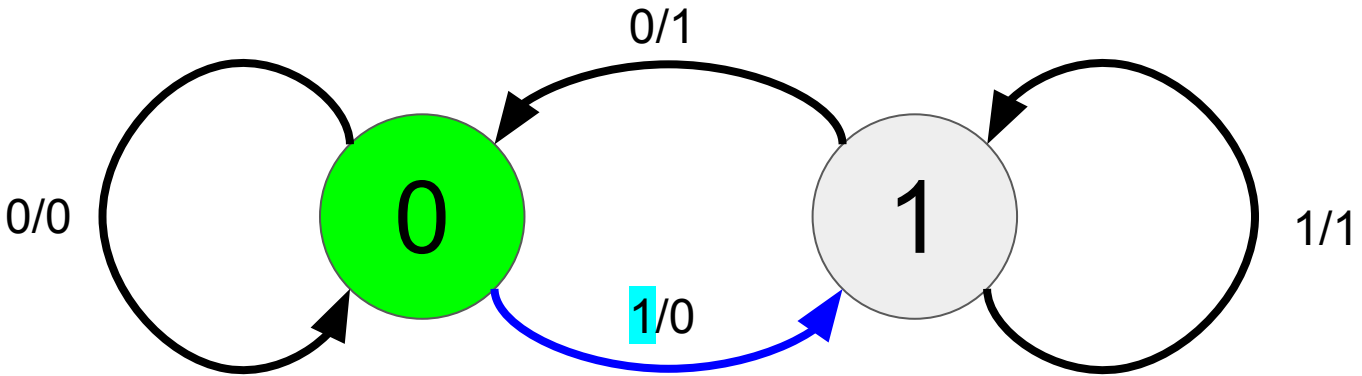
Finite State Machines

Example: Say we start at state 0, and send this input:
0b01101101101110111000010110
What is the resulting output?
0b0



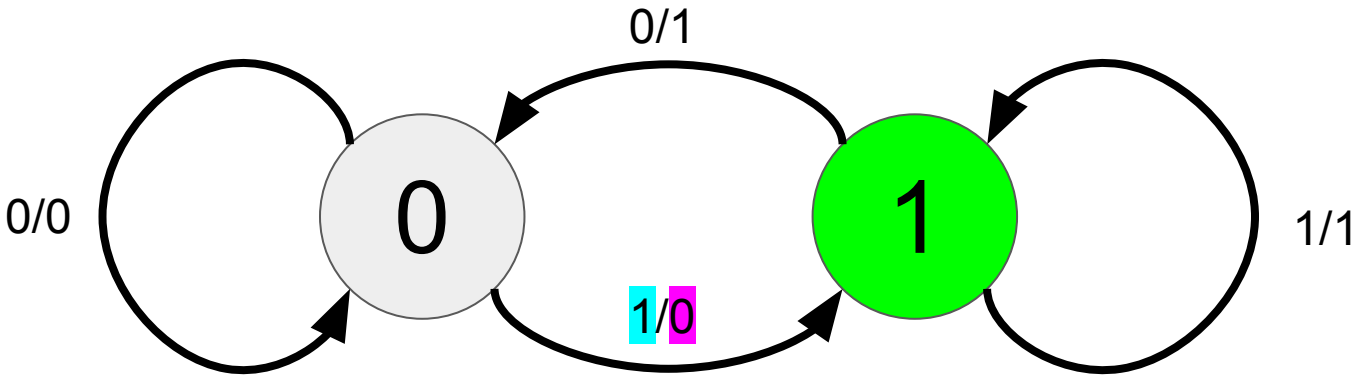
Finite State Machines

Example: Say we start at state 0, and send this input:
0b011011011011011100010110
What is the resulting output?
0b0



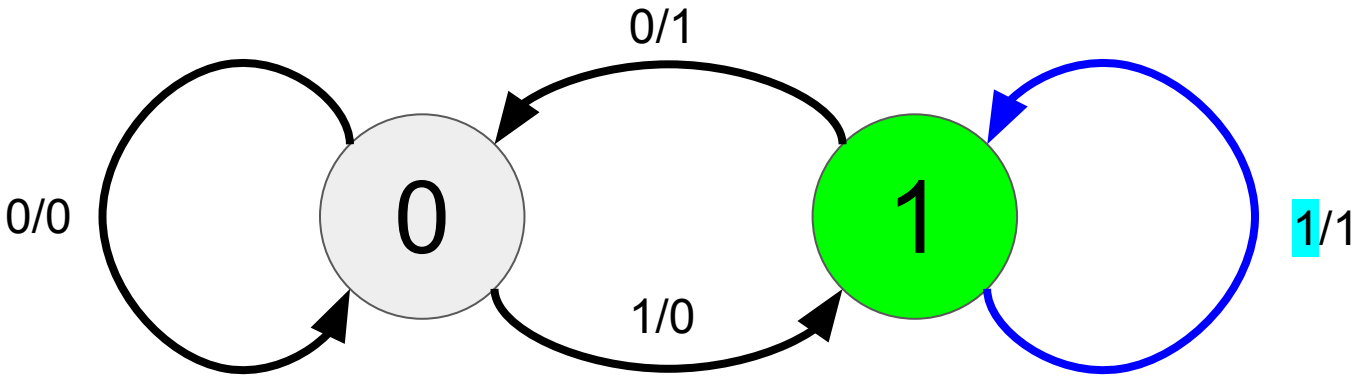
Finite State Machines

Example: Say we start at state 0, and send this input:
0b01101101101110111000010110
What is the resulting output?
0b00



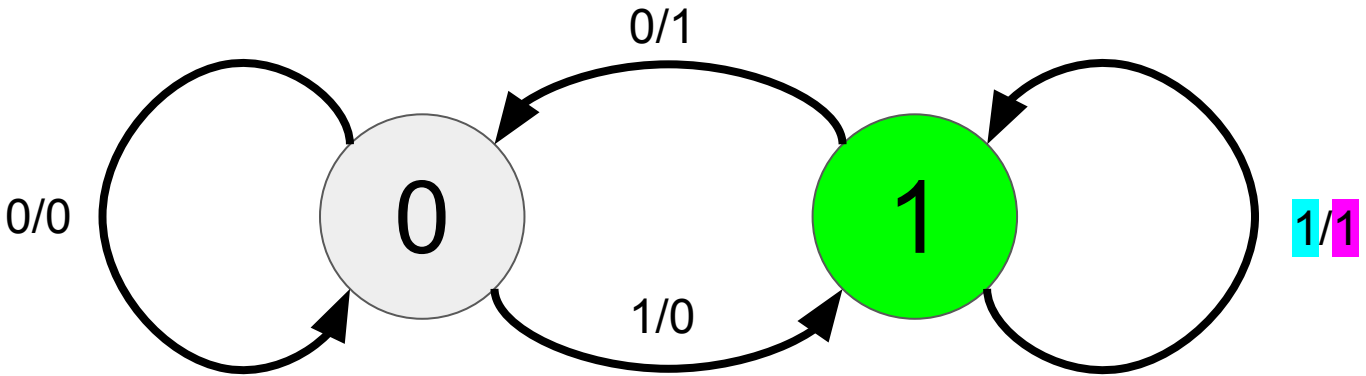
Finite State Machines

Example: Say we start at state 0, and send this input:
0b0110110110111011100010110
What is the resulting output?
0b00



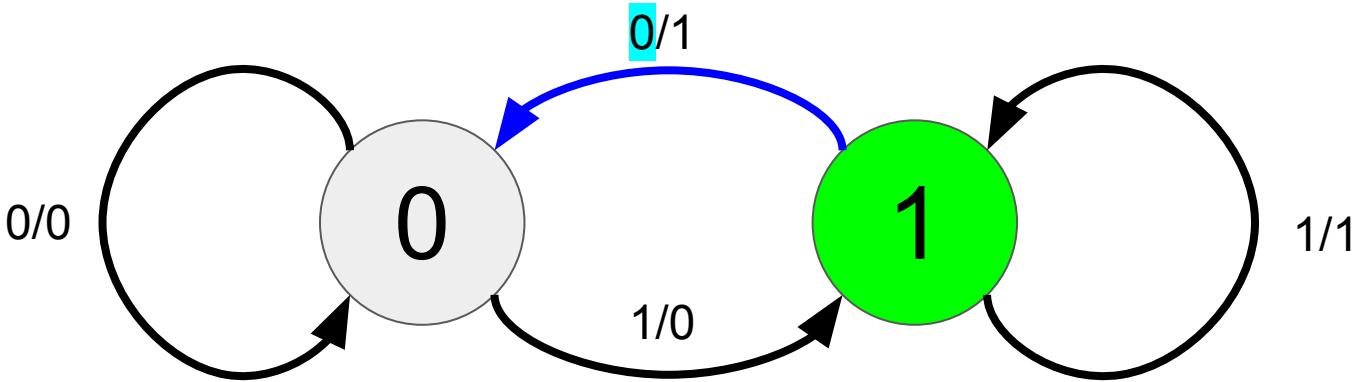
Finite State Machines

Example: Say we start at state 0, and send this input:
0b011011011011011100010110
What is the resulting output?
0b0011



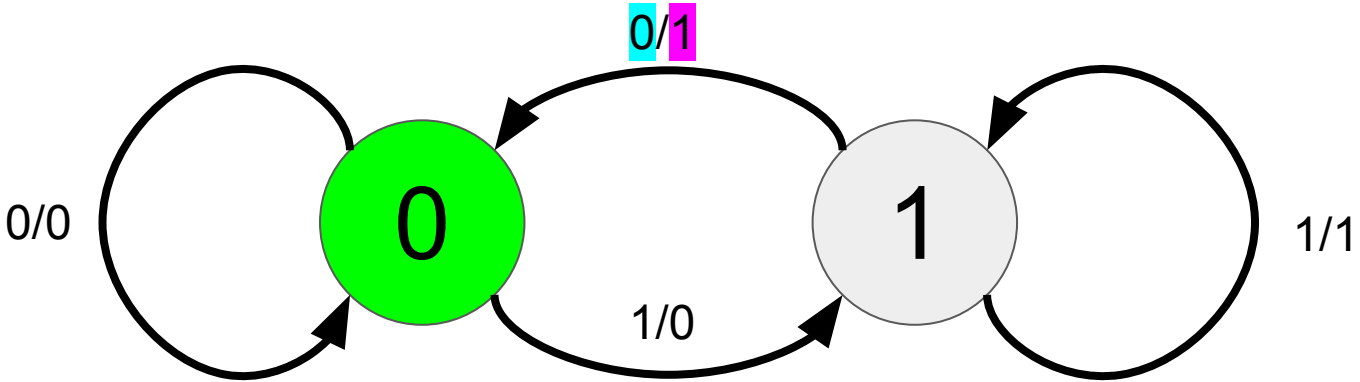
Finite State Machines

Example: Say we start at state 0, and send this input:
0b01101101101110111001010110
What is the resulting output?
0b001



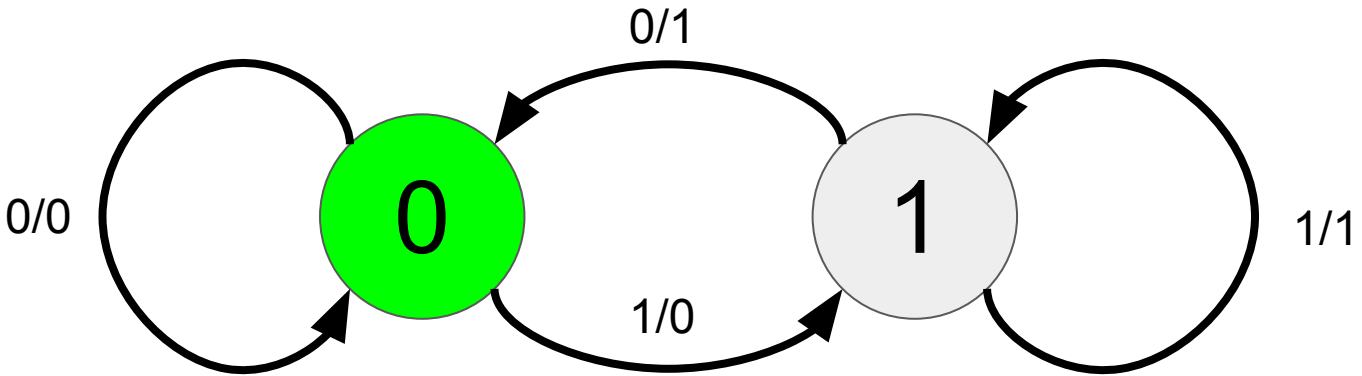
Finite State Machines

Example: Say we start at state 0, and send this input:
0b01101101101110111001010110
What is the resulting output?
0b00111



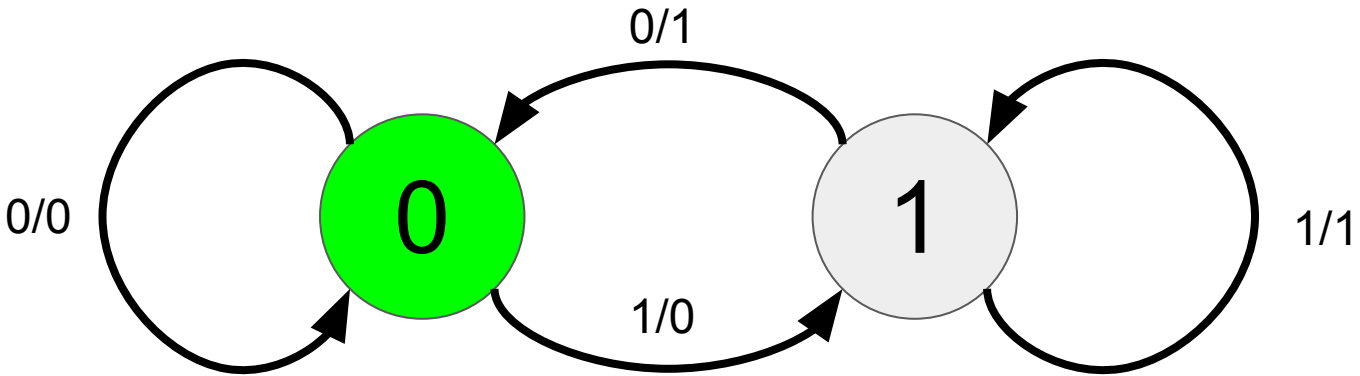
Finite State Machines

Example: Say we start at state 0, and send this input:
0b01101101101110111100010110
What is the resulting output?
0b00110110110111011110001011



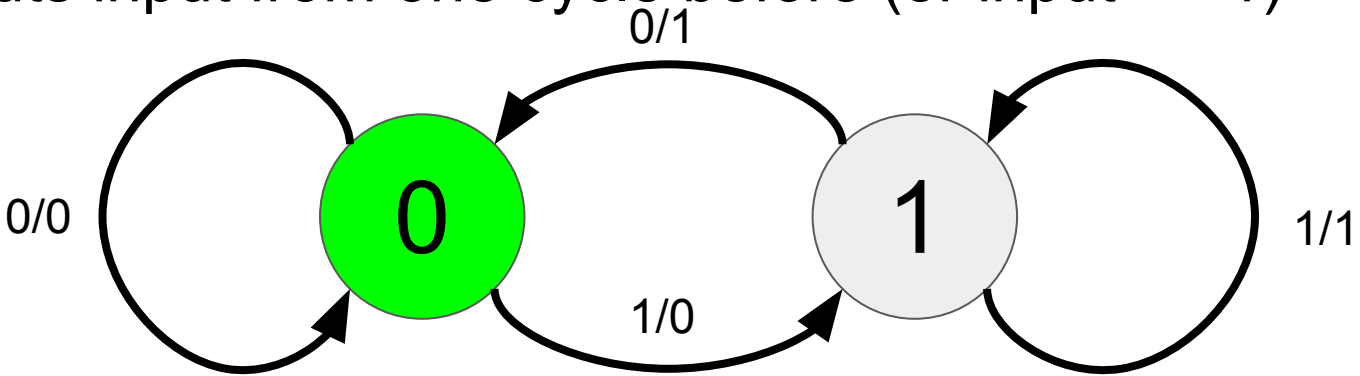
Finite State Machines

Example: Say we start at state 0, and send this input:
0b01101101101110111100010110
What is the resulting output?
0b00110110110111011110001011
What does this FSM compute?



Finite State Machines

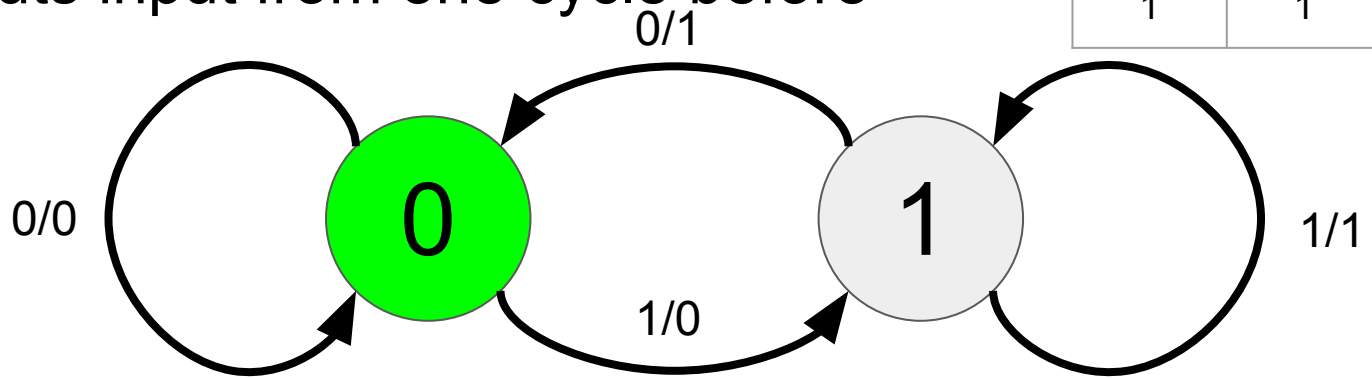
Example: Say we start at state 0, and send this input:
0b01101101101110111100010110
What is the resulting output?
0b00110110110111011110001011
What does this FSM compute?
Outputs input from one cycle before (or input >> 1)



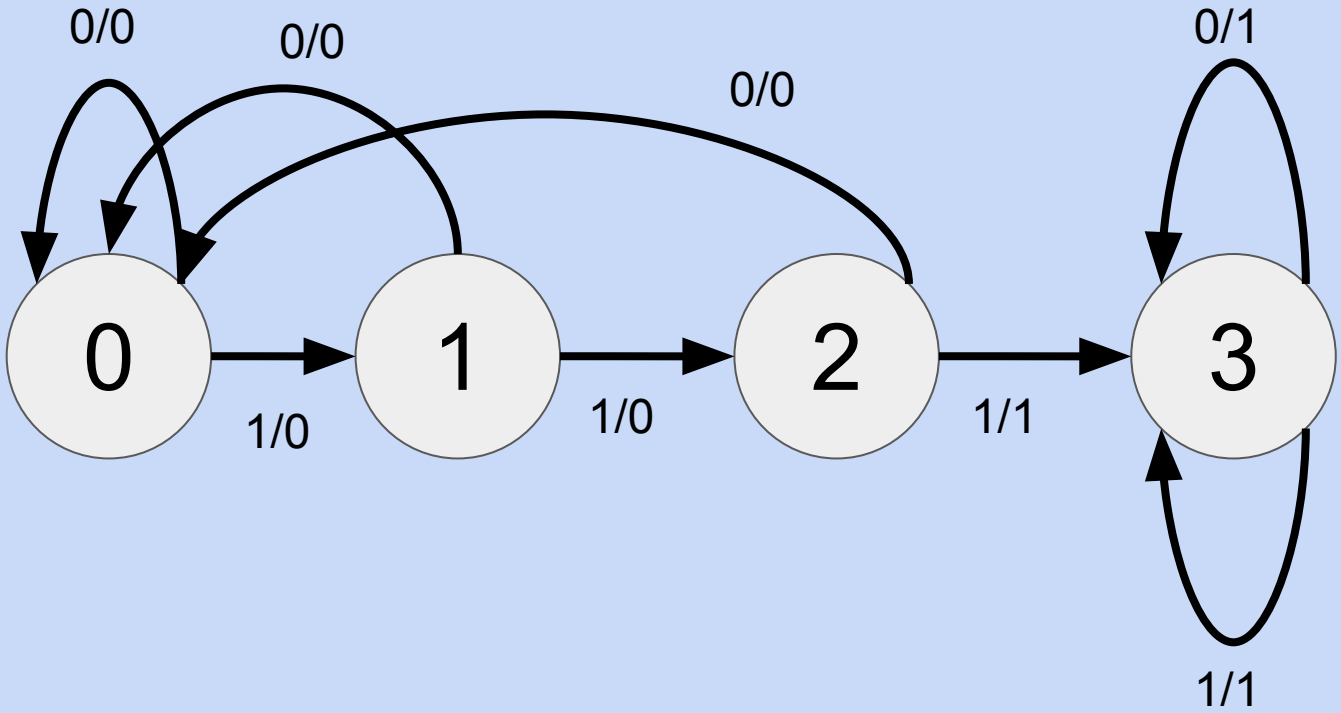
Finite State Machines

Say we start at state 0, and send this:
0b01101101101110111100010110
What is the resulting output?
0b00110110110111011110001011
What does this FSM compute?
Outputs input from one cycle before

State	Input	Output	Next
0	0	0	0
0	1	0	1
1	0	1	0
1	1	1	1



What does this FSM do?



Draw a FSM

Draw a FSM that outputs 1 precisely when the most recent two bits are different.
Example:

```
Input:  011011011011010000010110
Output: 010110110110111000011101
```

Draw a FSM (Hard)

Draw a FSM that, when given the bitwise representation of `N`, outputs the bitwise representation of `N//5`

Example:

```
Input:  01101101101101000010110 = 3594774
Output: 00010101111100001101010 =  718954
```

Why are FSMs relevant?

- A RISC-V Processor is an FSM!
  - States?
    - N-tuple of registers and memory ($2^{32*32+\text{num bits in memory}}$ states)
  - Inputs?
    - 32-bit instruction (one per clock cycle)
    - FSMs can receive inputs that aren't just bits!
  - Outputs?
    - Print output, otherwise output ignored

Why are FSMs relevant?

- An FSM can be constructed using a circuit!
  - Main approach:
    - At start of clock cycle, send input (somehow)
    - Compute output and new state from input and current state (somehow)
    - Save new state in state elements (somehow)
    - Repeat every clock cycle
  - Combinational logic can be used to compute next state and next output given input and current state
    - Worst-case: Use SOP (though we can do way better)
  - Need a way to "save" what the current state is
  - Need a way to periodically start new clock cycles
    - Tomorrow!